JDBC

JAVA PROGRAMMING – CSE1007

Student Management System

↓

Java

↓

JDBC

↓

Database

↓

MySQL

JDBC is a **Java API used to connect Java applications with databases and execute SQL statements**.

JDBC is not a database. It is a communication mechanism/API between Java and a database.

Register No	Name	Department	CGPA
101	Arun	CSE	8.5
102	Priya	IT	9.1
103	Ravi	ECE	7.8

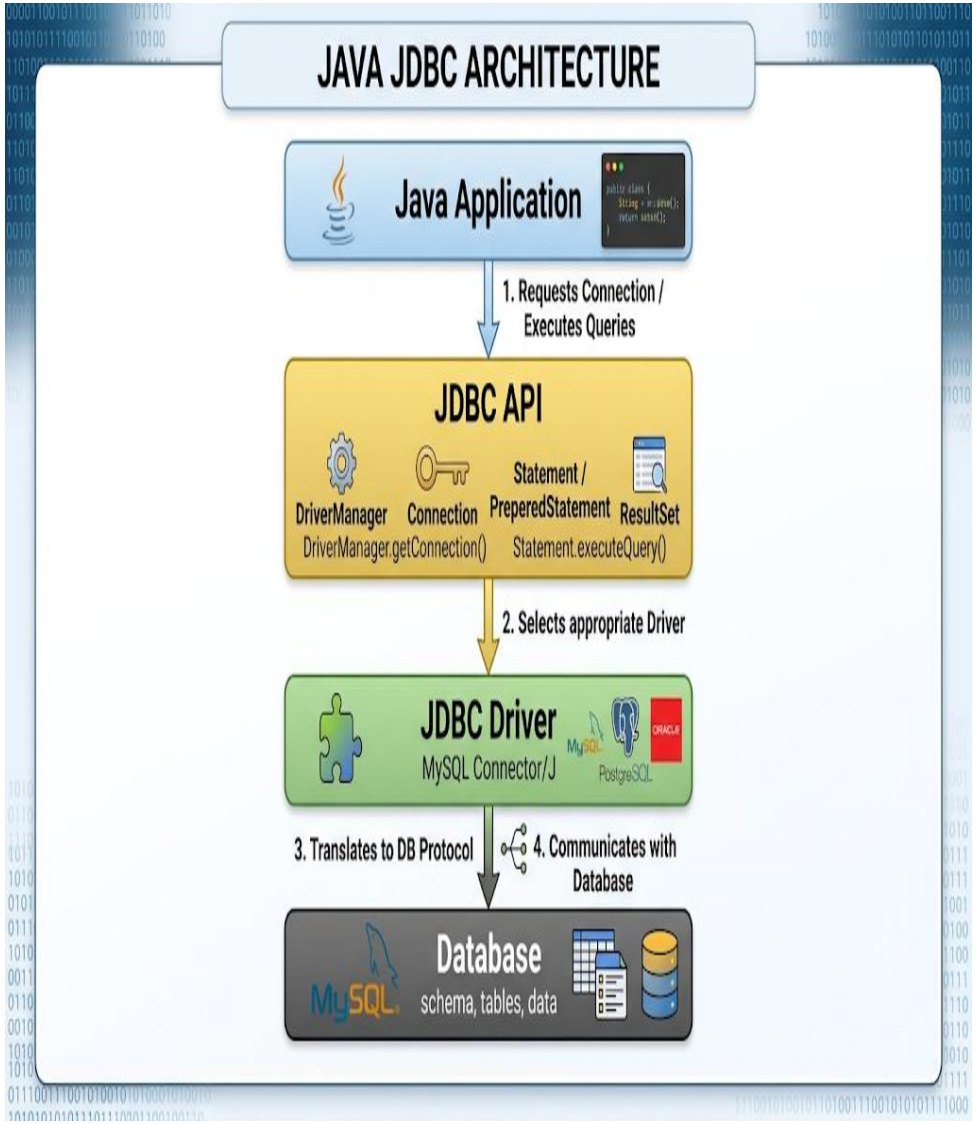
Java is responsible for the **application logic and user interface**.
The database is responsible for **storing and managing the data**.

Java Application
↓
JDBC
↓
JDBC Driver
↓
MySQL

Different databases use different technologies.

The JDBC API gives Java a **standard programming interface**.

Connection
Statement
PreparedStatement
ResultSet



Java Application
The program written by the developer.
JDBC API
Provides Java classes/interfaces for database operations.
JDBC Driver
Acts as a translator between Java/JDBC and the particular database.
Database
Stores the actual data.

For a basic MySQL JDBC program:

Required

1. JDK
2. Java IDE – Eclipse/IntelliJ/NetBeans
3. MySQL Server
4. MySQL database
5. MySQL JDBC Driver – Connector/J

8. Create a Database

```
CREATE DATABASE college;

USE college;

CREATE TABLE student (
   id INT PRIMARY KEY,
   name VARCHAR(50),
   department VARCHAR(30),
   cgpa DOUBLE
);
```

Insert sample data:

```
INSERT INTO student VALUES
(101, 'Arun', 'CSE', 8.5),
(102, 'Priya', 'IT', 9.1),
(103, 'Ravi', 'ECE', 7.8);
```

9. JDBC Programming Steps

1. Import JDBC packages
   ↓
2. Load/Register Driver
   ↓
3. Establish Connection
   ↓
4. Create Statement
   ↓
5. Execute SQL
   ↓
6. Process Result
   ↓
7. Close Resources

Modern JDBC drivers can often be auto-loaded, so explicit driver loading is generally unnecessary.

```
import java.sql.*;
```

This provides access to classes/interfaces such as:

Connection
Statement
PreparedStatement
ResultSet
SQLException
DriverManager

Establish Database Connection

```
Connection con = DriverManager.getConnection(
   "jdbc:mysql://localhost:3306/college",
   "root",
   "password"
);
```

URL

```
jdbc:mysql://localhost:3306/college
```

jdbc → JDBC protocol
mysql → database type
localhost → database server
3306 → MySQL port
college → database name

Connection Object

```
Connection con
```

The Connection object represents the connection between:

Java Application ↔ Database

Connection = communication channel between Java and the database.

Statement

Once connected, Java needs a way to send SQL commands.

```
Statement stmt = con.createStatement();

ResultSet rs = stmt.executeQuery(
   "SELECT * FROM student"
);
```

ResultSet
ResultSet represents the rows returned by a query.

executeQuery()

for SQL statements that **return data**

execute() can be used when the SQL statement may produce different types of results.

Method	Typical use
executeQuery()	SELECT
executeUpdate()	INSERT, UPDATE, DELETE
execute()	General-purpose execution

```
import java.sql.*;

public class StudentDatabase {

   public static void main(String[] args) {

      String url = "jdbc:mysql://localhost:3306/college";
      String username = "root";
      String password = "password";

      try {

         Connection con =
            DriverManager.getConnection(url, username,
password);

         Statement stmt = con.createStatement();

         ResultSet rs =
            stmt.executeQuery("SELECT * FROM student");
```

try → do database work
catch → handle error
close → close database connection

```
         while (rs.next()) {

            System.out.println(
               rs.getInt("id") + " " +
               rs.getString("name") + " " +
               rs.getString("department") + " " +
               rs.getDouble("cgpa")
            );
         }

         con.close();

      } catch (SQLException e) {

         e.printStackTrace();
      }
   }
}
```

C → Create
R → Read
U → Update
D → Delete

```
Statement stmt = con.createStatement();

int rows = stmt.executeUpdate(
   "INSERT INTO student VALUES " +
   "(104, 'Kumar', 'CSE', 8.2)"
);

System.out.println(rows + " row inserted");
```

INSERT

```
int rows = stmt.executeUpdate(
   "UPDATE student SET cgpa = 8.8 WHERE id = 104"
);
```

UPDATE

Suppose the user enters:

Student ID = 105
Name = Karthik
Department = IT
CGPA = 8.7

A beginner might construct SQL using string concatenation.

```
String sql =
   "INSERT INTO student VALUES (" +
   id + ", '" + name + "', '" +
   department + "', " + cgpa + ")";
```

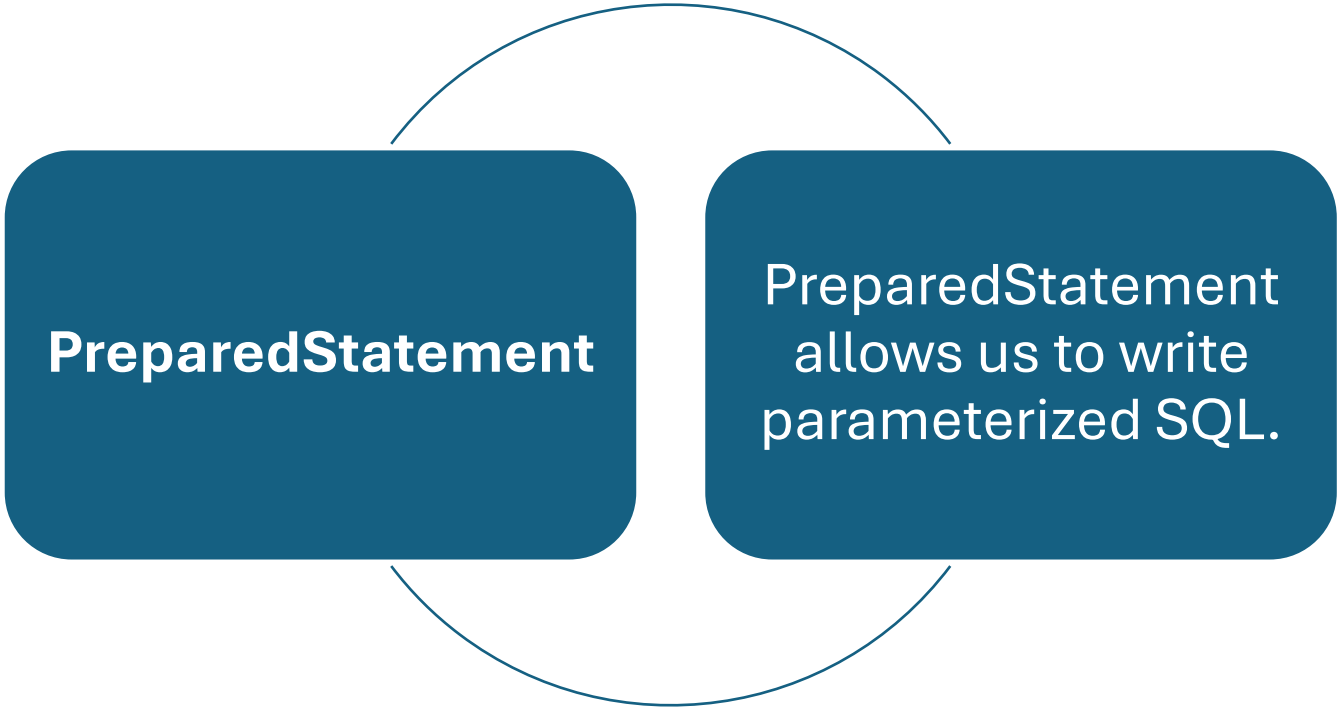
This approach can lead to problems, especially **SQL injection** when user-controlled text is inserted directly into SQL.

```
String sql =
   "INSERT INTO student VALUES (?, ?, ?, ?)";

PreparedStatement ps =
   con.prepareStatement(sql);
   ps.setInt(1, 104);
   ps.setString(2, "Kumar");
   ps.setString(3, "CSE");
   ps.setDouble(4, 8.2);


   ps.executeUpdate();
```

Values are supplied separately:



ResultSet is used to **read data returned by a SELECT query**.

```
String sql =
   "SELECT * FROM student";

PreparedStatement ps =
   con.prepareStatement(sql);

ResultSet rs =
   ps.executeQuery();

while (rs.next()) {

   System.out.println(
      rs.getInt("id") + " " +
      rs.getString("name") + " " +
      rs.getString("department")
   );
}
```

- executeQuery() → gets the result from the database
- ResultSet rs → stores the returned rows
- rs.next() → moves to the next row
- rs.getInt("id") → reads the id
- rs.getString("name") → reads the name

Output

If the database contains this record:

104 Kumar CSE
105 Ravi IT
106 Priya ECE

The program output will be:

104 Kumar CSE
105 Ravi IT
106 Priya ECE

ResultSet reads **one row at a time** using rs.next() and prints the values.

PreparedStatement → sends the SQL
ResultSet → receives/reads the SELECT result